Declaration of Use for Generative AI in Assessments

Generative AI tools in the following ways:

Acknowledgement of Generative AI Tools Used

- ☐ For brainstorming,
- ☐ To find sources.
- ☑ To plan the structure/outline of the work.
- ☑ To generate programming code.
- ☐ To generate translations of primary/secondary source content for consultation.
- ☐ To generate translations included in the submitted work, whether or not manually revised.
- ☐ To improve the language of my own phrases, sentences, and/or paragraphs.
- ☐ To generate the text of (part of) the submitted work.

Acknowledgment of Assessment Submission

I, **Reynerio Samos**, hereby confirm that on **08/19/2026**:

1. I am the author of this submitted document.
2. I am responsible for any AI-generated errors or fabrications.
3. I understand the limitations and risks of using AI.
4. I used AI tools ethically, protecting all sensitive information.
5. I ensure any AI-assisted work remains originally my own.
6. I have appropriately acknowledged all use of generative AI.
7. Undeclared use of AI constitutes academic dishonesty, which I acknowledge.
8. I am accountable for any resulting academic misconduct.

Please add more rows to the table as needed so that each tool used in the creation of your assessment submission is included in the declaration.

Generative AI Tool Used *(Please List Each Separately)*	**Purpose of Use**	**Briefly Explain the Extent of Use**
Claude - Sonnet 5	Code Correction and Planning	To fix errors in my starter code and help guide steps to structure the lab deliverable (in the submitted code).

Student Name: Reynerio Samos

Student Id: 2018119235

Course Code: CMPS 4191

Department: Faculty of Science and Technology

Signature: 

Date: **Aug 19, 2026**

Declaration of Use for Generative AI in Assessments

Generative AI tools in the following ways:

Acknowledgement of Generative AI Tools Used

- ☐ For brainstorming,
- ☐ To find sources.
- ☑ To plan the structure/outline of the work.
- ☑ To generate programming code.
- ☐ To generate translations of primary/secondary source content for consultation.
- ☐ To generate translations included in the submitted work, whether or not manually revised.
- ☐ To improve the language of my own phrases, sentences, and/or paragraphs.
- ☐ To generate the text of (part of) the submitted work.

Acknowledgment of Assessment Submission

I, **Chahiim Pop**, hereby confirm that on **08/19/2026**:

9. I am the author of this submitted document.
10. I am responsible for any AI-generated errors or fabrications.
11. I understand the limitations and risks of using AI.
12. I used AI tools ethically, protecting all sensitive information.
13. I ensure any AI-assisted work remains originally my own.
14. I have appropriately acknowledged all use of generative AI.
15. Undeclared use of AI constitutes academic dishonesty, which I acknowledge.
16. I am accountable for any resulting academic misconduct.

Please add more rows to the table as needed so that each tool used in the creation of your assessment submission is included in the declaration.

Generative AI Tool Used *(Please List Each Separately)*	**Purpose of Use**	**Briefly Explain the Extent of Use**
Claude	Code Correction & Doc assistant	Help troubleshoot errors and to understand unknown topics

Student Name: Reynerio Samos

Student Id: 2018119235

Course Code: CMPS 4191

Department: Faculty of Science and Technology

Signature: 

Date: **Aug 19, 2026**

CMPS4191 • LABORATORY 1 - Reynerio Samos & Chahiim Pop

Measuring a Synchronous API Contract

An empirical investigation of request lifetime, work duration, and client waiting time

Duration	Work mode	Code
75 minutes	Pairs	Provided

Central question: How does report-generation time affect the externally observable behavior of a synchronous report API?

Learning outcomes

- Measure API response time under controlled conditions.
- Distinguish a prediction, an observation, and a causal explanation.
- Identify the relationship between work duration and response time.
- Explain how the synchronous contract couples request lifetime and work lifetime.
- Derive a system requirement from measured evidence.

The current contract

A contract is an externally observable promise between components. It states what a client may expect from a server; it does not describe the server's internal implementation.

Current promise If the request succeeds, POST /v1/reports returns 200 OK with the completed report in the response body.

Because the completed report must be placed in the response, the handler must finish report generation before it can fulfill the contract. The client therefore waits while the work is performed.

Approximate relationship:

response time ≈ HTTP overhead + report-generation time

Experimental design

Research question

How does report-generation time affect the externally observable behavior of the synchronous report endpoint?

Hypothesis — complete before testing

If report-generation time increases, then the observable completion time of the endpoint increases because it's a synchronous endpoint, meaning all actions must be settled before a response is sent.

Variables

Role	Variable
Independent	Artificial report-generation delay: 0 s, 3 s, 7 s, 12 s
Dependent	Response time, HTTP status, response bytes, complete report received, server completion log
Controlled	Endpoint, request body, consumer, reporting period, database contents, client command and server timeout

Before you measure: make predictions

Record predictions before running the server under each condition. Predictions are not graded for being correct; they are evidence of your reasoning before observation.

Delay	Predicted response time	Predicted status	Complete body?
0 s	0<x<1s	OK - 200	Yes
3 s	3s < x < 4s	OK - 200	Yes
7 s	7s < x < 8s	OK - 200	Yes
12 s	12s < x < 13s	OK - 200	Yes

At what point do you predict the current design will fail or become unreliable? Why?

When the Delay is greater than a couple of minutes (greater than 2 mins) as the server will more than likely drop the connection as the time to respond is too great.

Procedure

Part A — Start one experimental condition

Your instructor will provide the database connection value. Start the API with one artificial-delay condition:

```
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=3s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=7s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=12s
```

Controlled experiment Change only the report-delay value. Restart the server between conditions. Keep the request, data and measurement command unchanged.

Part B — Send and measure the request

In a second terminal, create request.json once:

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2027-01-01T00:00:00Z"
}
```

Then run the same measurement command for every condition:

```
curl --silent --show-error \
  --output response.json \
  --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n' \
  --request POST \
  --header 'Content-Type: application/json' \
  --data @request.json \
  http://localhost:4000/v1/reports
```

After every request, inspect the body:

```
cat response.json
```

Part C — Repeat consistently

1. Run the request three times for the current delay.
2. Record every result; do not discard unexpected results.
3. Inspect response.json and the server log after each request.
4. Stop and restart the server with the next delay.
5. Do not run different delay conditions simultaneously.

Raw measurements

Record the client-visible result and the server-visible result. A blank body, status 000, timeout or connection error is still an observation and must be recorded.

Condition: 0 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	0.003402s	318	Yes	Yes
2	200	0.006068s	319	Yes	Yes
3	200	0.001293s	319	Yes	Yes

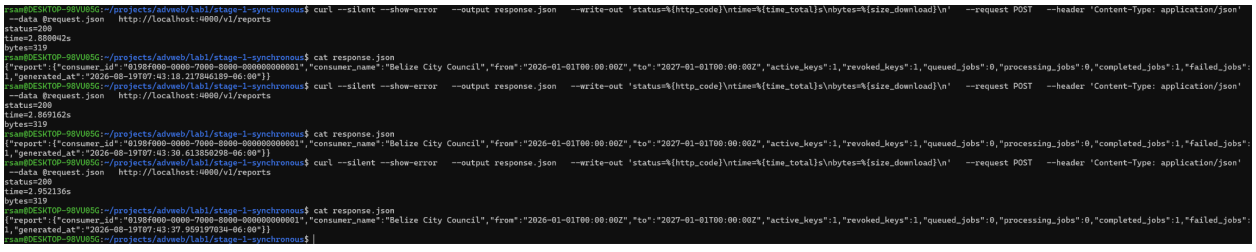
response.json Inspection:

Server Response:

Condition: 3 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	2.880042s	319	Yes	Yes
2	200	2.869162s	319	Yes	Yes
3	200	2.952136s	319	Yes	Yes

response.json inspection:



Server Response:

```
[2]-  Terminated              go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
rsam@DESKTOP-98VU05G:~/projects/advweb/lab1/stage-1-synchronous$ go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=3s
time=2026-08-19T07:42:56.617-06:00 level=INFO msg="database connection pool established"
time=2026-08-19T07:42:56.617-06:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-08-19T07:43:15.202-06:00 level=INFO msg="report generation started" consumer_id=0198f000-0000-7000-8000-000000000001 artificial_delay=3s
time=2026-08-19T07:43:18.220-06:00 level=INFO msg="report generation finished" consumer_id=0198f000-0000-7000-8000-000000000001
time=2026-08-19T07:43:29.117-06:00 level=INFO msg="report generation started" consumer_id=0198f000-0000-7000-8000-000000000001 artificial_delay=3s
time=2026-08-19T07:43:30.614-06:00 level=INFO msg="report generation finished" consumer_id=0198f000-0000-7000-8000-000000000001
time=2026-08-19T07:43:34.867-06:00 level=INFO msg="report generation started" consumer_id=0198f000-0000-7000-8000-000000000001 artificial_delay=3s
time=2026-08-19T07:43:37.960-06:00 level=INFO msg="report generation finished" consumer_id=0198f000-0000-7000-8000-000000000001
```

Condition: 7 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	6.691117s	319	Yes	Yes
2	200	6.698297s	319	Yes	Yes
3	200	6.696002s	319	Yes	Yes

Response.json inspection:

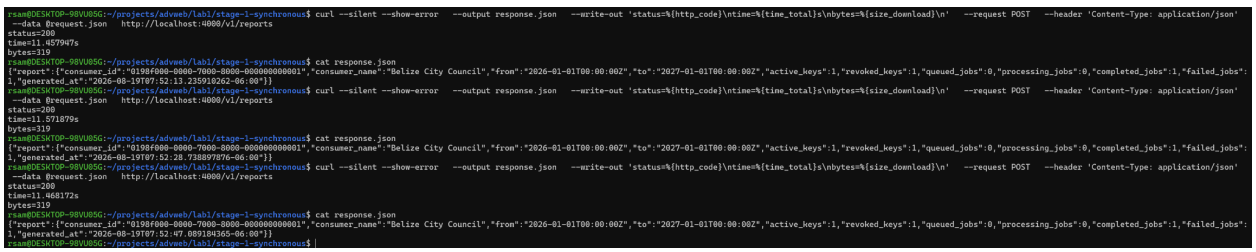
Server response:

```
rsam@DESKTOP-98VU05G:~/projects/advweb/lab1/stage-1-synchronous$ go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=7s
time=2026-08-19T07:48:06.434-06:00 level=INFO msg="database connection pool established"
time=2026-08-19T07:48:06.434-06:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-08-19T07:48:10.362-06:00 level=INFO msg="report generation started" consumer_id=0198f000-0000-7000-8000-000000000001 artificial_delay=7s
time=2026-08-19T07:48:17.368-06:00 level=INFO msg="report generation finished" consumer_id=0198f000-0000-7000-8000-000000000001
time=2026-08-19T07:48:21.572-06:00 level=INFO msg="report generation started" consumer_id=0198f000-0000-7000-8000-000000000001 artificial_delay=7s
time=2026-08-19T07:48:28.611-06:00 level=INFO msg="report generation finished" consumer_id=0198f000-0000-7000-8000-000000000001
time=2026-08-19T07:48:35.940-06:00 level=INFO msg="report generation started" consumer_id=0198f000-0000-7000-8000-000000000001 artificial_delay=7s
time=2026-08-19T07:48:41.445-06:00 level=INFO msg="report generation finished" consumer_id=0198f000-0000-7000-8000-000000000001
```

Condition: 12 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	11.457947s	319	Yes	Yes
2	200	11.571879s	319	Yes	Yes
3	200	11.468172s	319	Yes	Yes

Response.json inspection:



Server response:

Summarize the evidence

For each condition, calculate the approximate mean response time:

$$\text{mean response time} = (T_1 + T_2 + T_3) \div 3$$

Delay	Mean response time	Successful responses	Complete reports
0 s	≈ 0.003588s	3 / 3	3 / 3
3 s	≈ 2.900447s	3 / 3	3 / 3
7 s	≈ 6.695139s	3 / 3	3 / 3
12 s	≈ 11.499333s	3 / 3	3 / 3

Observation

Write only what you measured. Do not explain the mechanism yet.

When report delay increased from 0s to 12s, mean response time changed from approximately 0.003s to 11.5s.

This is similar to the predictions made with the caveat that the lower bound was predicted to be more than the delay time, when in reality it is slightly less than the delay time.

Explanation

Now explain the mechanism that produced the observations.

Why did response time change as the artificial delay increased?

The response time changes as delay increases because the report generation and response is done synchronously, meaning all previous instructions must be cleared before the next can proceed including the timer that adds delay (reportDelay).

What must the handler complete before it can send 200 OK with the report?

The handler must do the previous instructions synchronously then query the database in Reports.Generate(), and when that query returns a successful response into report, the handler returns the 200 OK status.

While the report is being generated, is the original request complete or outstanding?

The original request is outstanding as the HTTP request is still open while the report is being generated. As the curl process is waiting on the response.json from the server which must generate the report and form it into that json, which won't return the 200 OK status (meaning it's complete) until it's settled.

Would asynchronous execution make the report faster, or would it change when the client receives acknowledgement?

Asynchronous execution won't necessarily make the report generate faster, but it would change when the client receives acknowledgement and give the impression that the report is going faster. As currently the client sends the request and the process hangs until it's completed. Where asynchronous execution would give the client acknowledgement that the process is executing and also allow other processes like the database connection to be opened before a request is sent.

Do not confuse cause with threshold A 10-second write timeout may expose the problem, but 10 seconds is not a universal boundary. Increasing the timeout moves the boundary; it does not remove the coupling.

From evidence to requirements

A requirement describes a property the system must provide. Do not write implementation mechanisms such as 202, worker, queue or polling in this section.

A valid report request should be acknowledged within 3 seconds.

Report generation should not depend on having the connection open or waiting for other processes to be settled that don't contribute to the generation of the report (such as closing the database connection).

The client should be able to determine the status of their request and changes in that status.

The system should preserve the result/response of the report generation even in the events of closed connections or timeouts.

Report-generation duration should not determine when the client is acknowledged of their request status.

The contract decision

The current contract promises completion in the original response. Based on your evidence and derived requirements, choose one option and justify it.

- ☐ Keep the current contract: return 200 OK with completed report
- ☑ Change the contract: acknowledge accepted work before the report is complete

Justification:

Based on the requirements listed, acknowledgement should be given to clients before work on the report is complete to let them know what status their request falls under, see the progression on that status and if it's safe to close the connection to check on their response later. This will require asynchronous execution to be implemented in the API to allow for this change in the contract.

In-class exit ticket

Submit one response per pair before leaving.

Element	Your response
Claim	The synchronous API is / is not suitable for potentially long-running reports because … The client must remain connected and wait for the full duration of the report, with no way to receive acknowledgement until report completion which grows with delay time.
Evidence	Provide at least two measured values. When the delay was 0s, the mean response time was 0.003s which is not too long of a wait. However when the delay was increased to 12s, mean response time increased to 11.5s, which means the client will not get acknowledgement until after 11.5s of sending a request.
Reasoning	Explain how the evidence follows from the current contract. The evidence follows directly from the contract where the handler has to fully generate a report before it can send a 200 OK response to the client. For this to happen the client must stay open for

Element	Your response
	that duration and end of that process, with the inability to acknowledge the request beforehand, making response time and report generation strongly correlated.
Requirement	State one requirement the current contract cannot reliably satisfy. A valid report request should be acknowledged independently of how long a report takes to generate, which the current contract cannot satisfy.

Complete outside class

Submit a short laboratory report containing the following sections:

- Research question and hypothesis
- Independent, dependent and controlled variables
- Raw measurements and calculated mean response times
- Observations stated without explanation
- Causal explanation of the observed behavior
- At least two limitations of the experiment
- Requirements derived from the evidence
- A proposed revised contract

Required reasoning chain

Layer	Question to answer
Evidence	What did we measure? We measured the time b/w responses of requests based on an artificial delay timer.
Problem	What coupling did the evidence expose? The evidence showed that report generation and response time are strongly coupled.
Requirement	What property does the system now need? The system needs the property to send responses irrespective of report generation.
Contract	What should the API promise the client? The API should promise the client a response before report generation is completed.
Implementation	What mechanisms might provide that contract? Asynchronous helpers might provide this contract.

Important distinctions

Incorrect conclusion	Stronger conclusion
Async makes report generation faster.	Async can separate acknowledgement time from completion time; the work may take just as long.
The design fails after exactly 10 seconds.	The observed threshold depends on configuration; the underlying issue is lifetime coupling.
The timeout is the whole problem.	A larger timeout preserves the synchronous contract and merely permits longer waiting.
202 means the report succeeded.	202 means the server accepted the work; completion or failure must be observed later.

Next laboratory You will redesign the contract using Job + Worker + 202 Accepted + GET /jobs/{id}, then repeat the measurements and compare acknowledgement time with completion time.